Spark 运行机制理解

第六题：结合 Spark WordCount 代码梳理 Spark 作业提交流程，代码级

1. 逐行说明 Spark WordCount 代码

```
import org.apache.spark.{SparkConf, SparkContext}

object WordCount {
  def main(args: Array[String]): Unit = {
    // ==================== 第 1 步：创建 SparkConf
====================
    // 作用：配置 Spark 应用程序的运行参数
    // 设置：应用名称、运行模式（local[*]表示本地模式，使用所有 CPU
核心）
    val conf = new SparkConf()
      .setAppName("WordCount")          // 应用名称，会在 Web
UI 显示
      .setMaster("local[*]")            // 本地模式，使用所有核
心

    // ==================== 第 2 步：创建 SparkContext
====================
    // 作用：Spark 应用程序的入口，负责初始化所有核心组件
    // 包括：DAGScheduler、TaskScheduler、SchedulerBackend
    val sc = new SparkContext(conf)

    // ==================== 第 3 步：读取数据创建 RDD
====================
```

```
    // 作用：从文件系统读取文本文件，创建初始 RDD
    // textFile 方法会创建 HadoopRDD，然后通过 MapPartitionsRDD
包装
    // 此时只是元数据记录，还未实际读取数据
    val lines: RDD[String] = sc.textFile("data/words.txt")

    // ==================== 第 4 步：flatMap 转换操作
====================
    // 作用：将每行文本按空格切分成单词，并将所有行的结果扁平化为一
个 RDD
    // 输入：一行字符串
    // 输出：多个单词
    // 示例："hello world" -> ["hello", "world"]
    val words: RDD[String] = lines.flatMap(_.split(" "))

    // ==================== 第 5 步：map 转换操作
====================
    // 作用：将每个单词映射为键值对形式 (word, 1)
    // 输入：单词字符串
    // 输出：(单词, 1) 元组
    // 示例："hello" -> ("hello", 1)
    val pairs: RDD[(String, Int)] = words.map(word => (word, 1))

    // ==================== 第 6 步：reduceByKey 行动操作
====================
    // 作用：根据 key 进行聚合，相同 key 的值相加
    // 这是一个宽依赖操作（Shuffle），会触发 Stage 划分
```

```
    // 输入：(word, 1) 的 RDD

    // 输出：(word, count) 的 RDD
    val wordCounts: RDD[(String, Int)] = pairs.reduceByKey(_ +
_)

    // ==================== 第 7 步：collect 行动操作
====================
    // 作用：将结果从集群收集到 Driver 端

    // 这是一个行动操作（Action），会触发真正的计算

    // 返回 Array[(String, Int)] 到 Driver 内存
    val result: Array[(String, Int)] = wordCounts.collect()

    // ==================== 第 8 步：打印结果
====================
    result.foreach(println)

    // ==================== 第 9 步：关闭 SparkContext
====================
    // 作用：释放资源，停止所有 Executor
    sc.stop()
  }
}
```

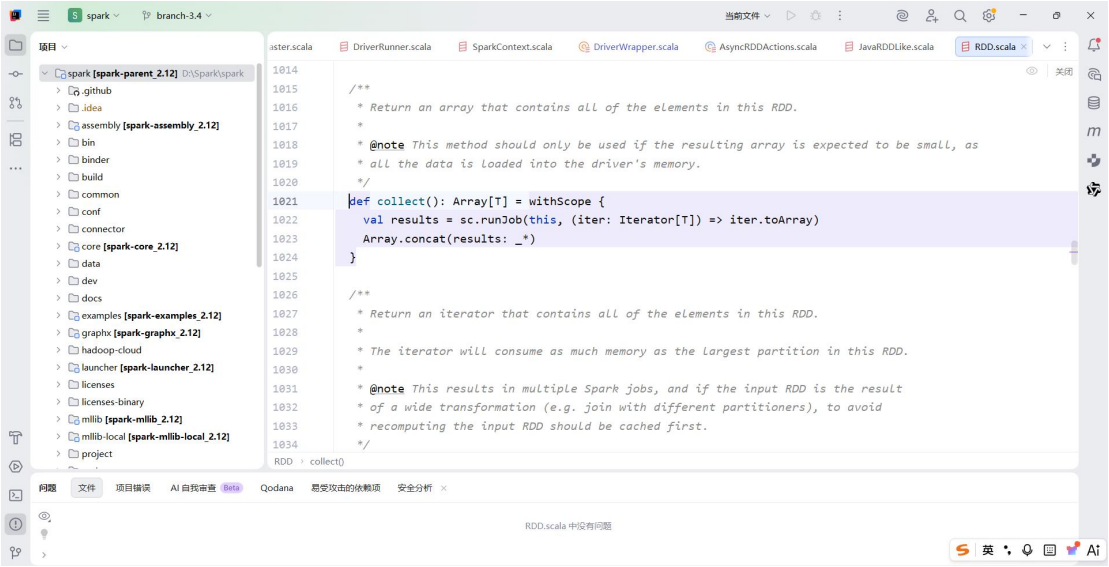
2. 梳理 Spark RunJob 作业链，顺着链条代码级梳理，Spark Intellij 源码分析截图

2.1 作业提交流程总览图

```
用户代码调用 collect()
        ↓
SparkContext.runJob()
        ↓
DAGScheduler.runJob()
        ↓
DAGScheduler.submitJob()
        ↓
DAGScheduler.handleJobSubmitted()
        ↓
DAGScheduler.newStage() (划分Stage)
        ↓
DAGScheduler.submitStage() (提交Stage)
        ↓
DAGScheduler.submitMissingTasks() (创建Task)
        ↓
TaskScheduler.submitTasks() (提交TaskSet)
        ↓
SchedulerBackend.reviveOffers() (资源调度)
        ↓
Executor.launchTask() (执行Task)
```

2.2 源码级逐层追踪

第 1 层：用户代码调用 collect()



第 2 层：SparkContext.runJob() 多重调用链

```
2238    * @param rdd target RDD to run tasks on
2239    * @param func a function to run on each partition of the RDD
2240    * @param partitions set of partitions to run on; some jobs may not want to compute on all
2241    * partitions of the target RDD, e.g. for operations like `first()`
2242    * @param resultHandler callback to pass each result to
2243    */
2244   def runJob[T, U: ClassTag](  Josh Rosen +11
2245       rdd: RDD[T],
2246       func: (TaskContext, Iterator[T]) => U,
2247       partitions: Seq[Int],
2248       resultHandler: (Int, U) => Unit): Unit = {
2249     if (stopped.get()) {
2250       throw new IllegalStateException("SparkContext has been shutdown")
2251     }
2252     val callSite = getCallSite
2253     val cleanedFunc = clean(func)
2254     logInfo("Starting job: " + callSite.shortForm)
2255     if (conf.getBoolean("spark.logLineage", false)) {
2256       logInfo("RDD's recursive dependencies:\n" + rdd.toDebugString)
```

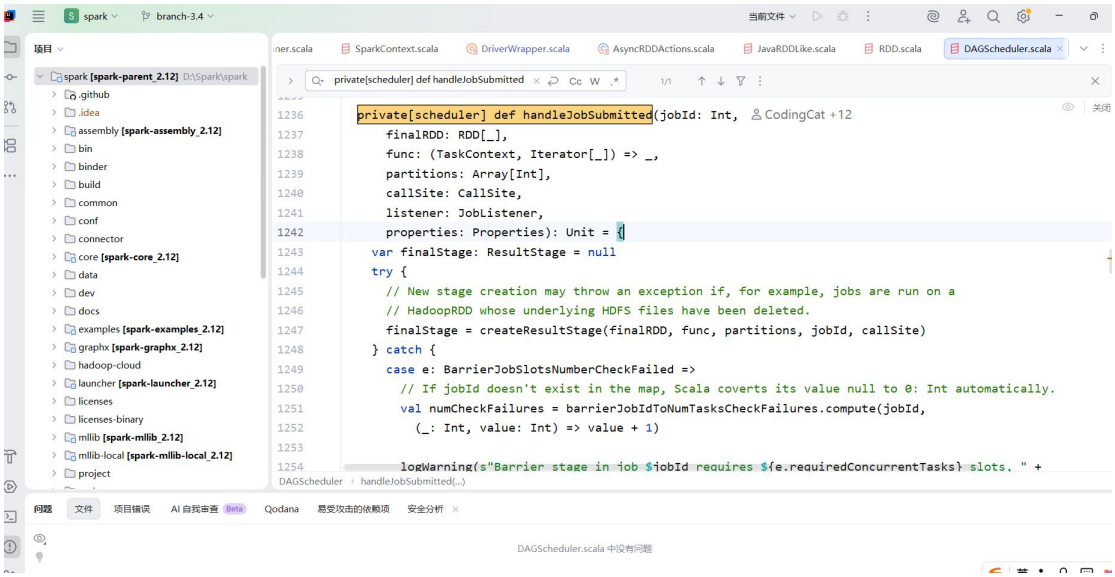
第 3 层：DAGScheduler.runJob() 提交作业

```
955     *
956     * @note Throws `Exception` when the job fails
957     */
958    def runJob[T, U](  Matei Zaharia +10
959        rdd: RDD[T],
960        func: (TaskContext, Iterator[T]) => U,
961        partitions: Seq[Int],
962        callSite: CallSite,
963        resultHandler: (Int, U) => Unit,
964        properties: Properties): Unit = {
965      val start = System.nanoTime
966      val waiter = submitJob(rdd, func, partitions, callSite, resultHandler, properties)
967      ThreadUtils.awaitReady(waiter.completionFuture, Duration.Inf)
968      waiter.completionFuture.value.get match {
969        case scala.util.Success(_) =>
970          logInfo("Job %d finished: %s, took %f s".format
971            (waiter.jobId, callSite.shortForm, (System.nanoTime - start) / 1e9))
972        case scala.util.Failure(exception) =>
973          logInfo("Job %d failed: %s, took %f s".format
```

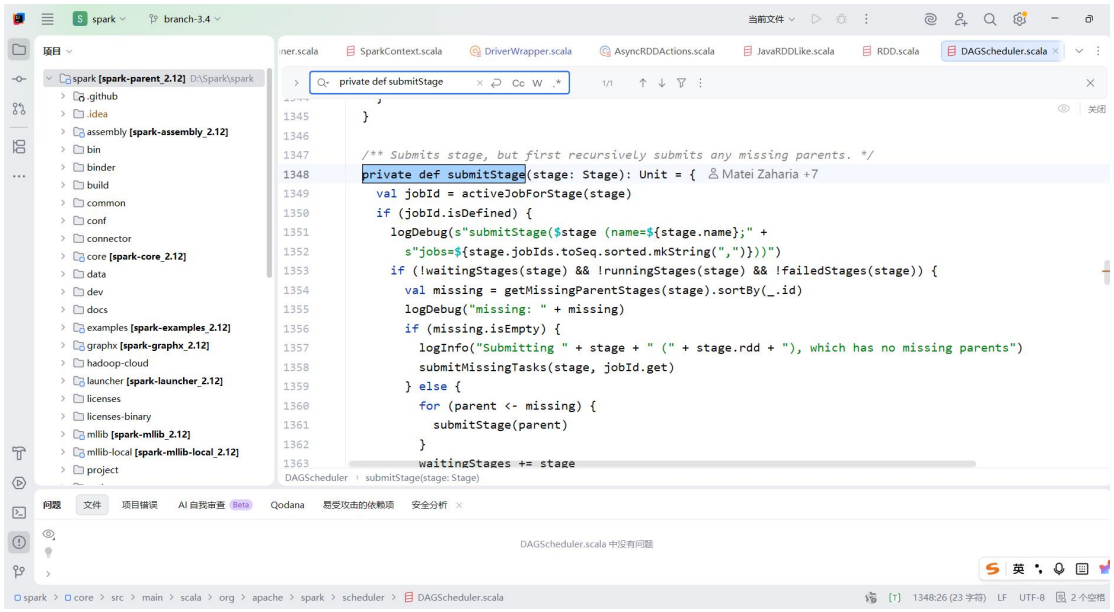
第 4 层：DAGScheduler.submitJob() 提交作业

```
900    def submitJob[T, U](  Reynold Xin +13
901        rdd: RDD[T],
902        func: (TaskContext, Iterator[T]) => U,
903        partitions: Seq[Int],
904        callSite: CallSite,
905        resultHandler: (Int, U) => Unit,
906        properties: Properties): JobWaiter[U] = {
907      // Check to make sure we are not launching a task on a partition that does not exist.
908      val maxPartitions = rdd.partitions.length
909      partitions.find(p => p >= maxPartitions || p < 0).foreach { p =>
910        throw new IllegalArgumentException(
911          "Attempting to access a non-existent partition: " + p + ". " +
912            "Total number of partitions: " + maxPartitions)
913      }
914
915      // SPARK-23626: `RDD.getPartitions()` can be slow, so we eagerly compute
916      // `.partitions` on every RDD in the DAG to ensure that `getPartitions()`
917      // is evaluated outside of the DAGScheduler's single-threaded event loop:
918      eagerlyComputePartitionsForRddAndAncestors(rdd)
```

第 5 层：DAGScheduler.handleJobSubmitted() 处理作业



第 6 层：Stage 划分与提交



第 7 层：创建并提交 TaskSet

```scala
private def submitMissingTasks(stage: Stage, jobId: Int): Unit = {
  logDebug("submitMissingTasks(" + stage + ")")

  // Before find missing partition, do the intermediate state clean work first.
  // The operation here can make sure for the partially completed intermediate stage,
  // `findMissingPartitions()` returns all partitions every time.
  stage match {
    case sms: ShuffleMapStage if stage.isIndeterminate && !sms.isAvailable =>
      mapOutputTracker.unregisterAllMapAndMergeOutput(sms.shuffleDep.shuffleId)
      sms.shuffleDep.newShuffleMergeState()
    case _ =>
  }

  // Figure out the indexes of partition ids to compute.
  val partitionsToCompute: Seq[Int] = stage.findMissingPartitions()
```

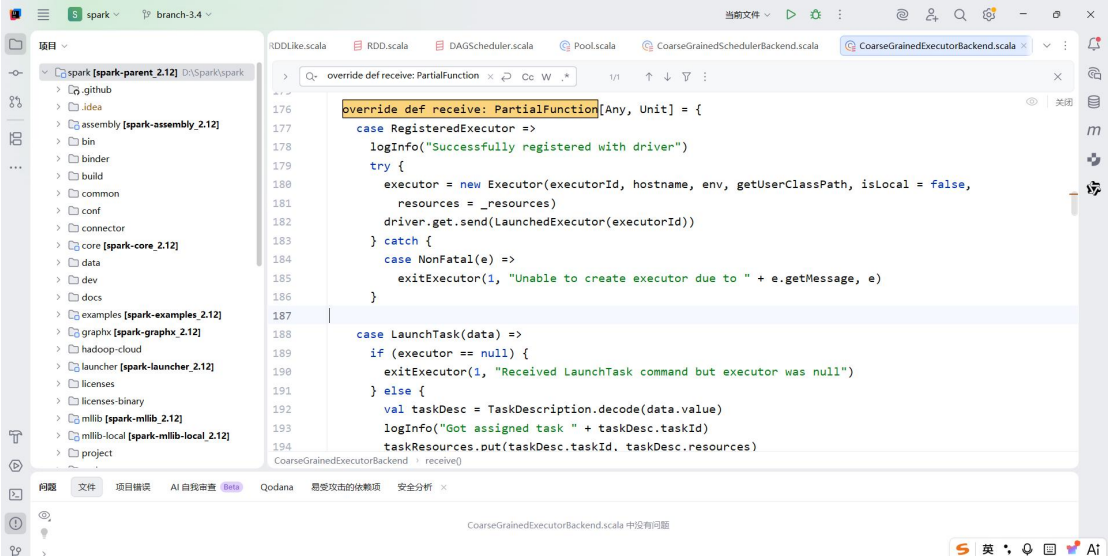
第 8 层：TaskSchedulerImpl 提交任务

```scala
override def submitTasks(taskSet: TaskSet): Unit = {
  val tasks = taskSet.tasks
  logInfo("Adding task set " + taskSet.id + " with " + tasks.length + " tasks "
    + "resource profile " + taskSet.resourceProfileId)
  this.synchronized {
    val manager = createTaskSetManager(taskSet, maxTaskFailures)
    val stage = taskSet.stageId
    val stageTaskSets =
      taskSetsByStageIdAndAttempt.getOrElseUpdate(stage, new HashMap[Int, TaskSetManager])

    // Mark all the existing TaskSetManagers of this stage as zombie, as we are adding a new one.
    // This is necessary to handle a corner case. Let's say a stage has 10 partitions and has 2
    // TaskSetManagers: TSM1(zombie) and TSM2(active). TSM1 has a running task for partition 10
    // and it completes. TSM2 finishes tasks for partition 1-9, and thinks he is still active
    // because partition 10 is not completed yet. However, DAGScheduler gets task completion
    // events for all the 10 partitions and thinks the stage is finished. If it's a shuffle stage
```

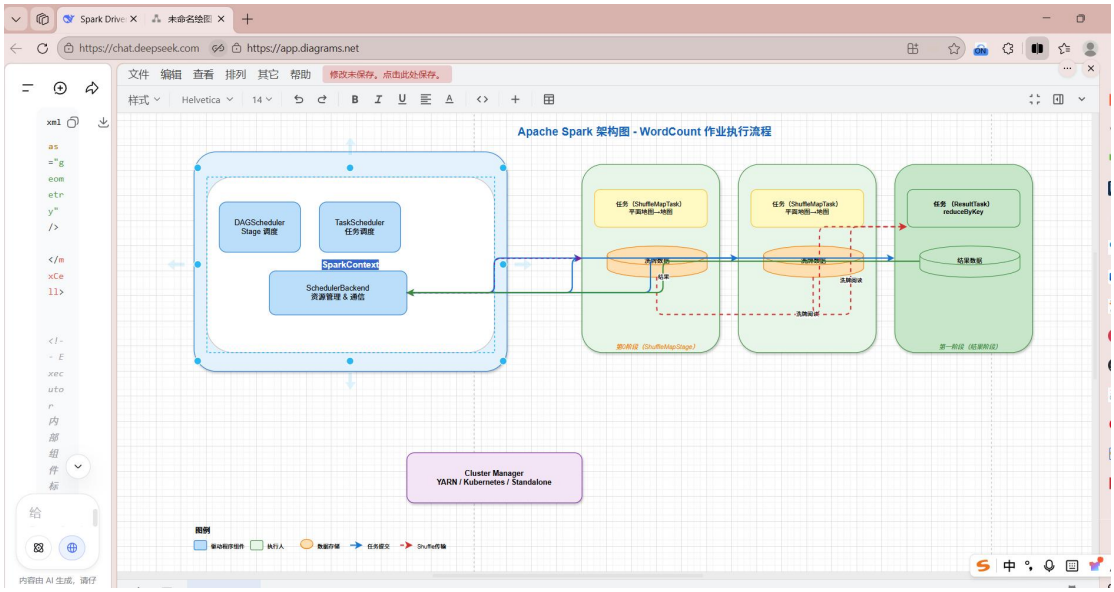
第 9 层：SchedulerBackend 资源调度

```scala
override def reviveOffers(): Unit = Utils.tryLogNonFatalError {
  driverEndpoint.send(ReviveOffers)
}

override def killTask(
    taskId: Long, executorId: String, interruptThread: Boolean, reason: String): Unit = {
  driverEndpoint.send(KillTask(taskId, executorId, interruptThread, reason))
}

override def defaultParallelism(): Int = {
  conf.getInt("spark.default.parallelism", math.max(totalCoreCount.get(), 2))
}

/**
 * Called by subclasses when notified of a lost worker. It just fires the message and returns
 * at once.
 */
protected def removeExecutor(executorId: String, reason: ExecutorLossReason): Unit = {
  driverEndpoint.send(RemoveExecutor(executorId, reason))
```

第 10 层：Executor 执行任务



3. 绘制 Spark 架构图，利用 draw.io 绘制



总结：

通过第六题以 **WordCount 为例逐层梳理了作业提交流程**，从用户代码调用 collect() 开始，经过 SparkContext.runJob()、DAGScheduler 的 Stage 划分与 TaskSet 创建、TaskScheduler 的任务分发，最终到 Executor 执行 Task 并返回结果，完整呈现了从 Action 触发到计算执行的全链路；最后通过 **draw.io 绘制了 Spark 架构图**，将 Driver、Executor、Cluster Manager 三大核心组件及其交互关系可视化呈现，并使用 XML 代码实现了图表的版本化管理和自动化生成。通过本次作业，深入理解了 Spark 的"Driver 协调 + Executor 执行"的分布式计算架构，以及 DAGScheduler 根据

宽窄依赖划分 Stage、TaskScheduler 进行任务调度的核心机制。